AI Foundation Training — Action Plan for 40 IT Students

Overview and Targets

This plan runs the 60-hour AI Foundation training for 40 IT students as 10 groups of 4, each group building one agricultural intelligence project in a single shared GitHub repository.

Item	Value
Learners	40 IT students (one cohort)
Groups	10, of 4 members each
Trainers	2, each supporting 5 groups
Course length	60 hours: 40 self-study + 20 workshop
Problem themes	10, each split into 4 subproblems
Learning method	Problem-based learning (PBL)
Repository	One central GitHub repository, one branch per group

By the end, each group delivers one integrated system, and the ten systems are merged into one course project.

Participants and Prerequisites

The 40 learners are IT students, so they bring computational skills and need to learn agricultural systems and operations.

Strength	Learning need	Expected capability
Computational skills	Agricultural systems and operations	Transform agricultural workflows into deployable AI workflows

IT learners can also go deeper on the technical side: data-processing code, statistical functions, building or extending agent capabilities, testing, and reproducibility.

Before day 1, learners should be able to:

- Use Git and GitHub at a basic level [confirm]
- Write basic Python [confirm]
- Work with CSV or Excel files [confirm]

Group Formation

The trainers form 10 balanced groups of 4 from the 40 students before the first workshop, so every group can cover all four member roles.

Proposed process:

1. Students complete the skills survey: programming, data and machine learning, documentation and communication, and Git experience.
2. Trainers place the 40 students into 10 groups of 4, mixing strengths so each group has at least one confident programmer and one confident writer [confirm criteria].
3. Trainers announce the groups and number them `group-01` to `group-10`, with each trainer supporting 5 groups.
4. Once the theme is assigned, each member takes one subproblem.

Groups stay fixed for the whole course, so each group's work builds up in one folder of the shared repository.

Problem Theme Assignment

Each group gets one main problem theme, divided into four related subproblems, one per member.

Group	Main problem theme	Four individual subproblems
1	Crop Pest Intelligence	Pest identification; pest monitoring; crop damage assessment; crop protection knowledge
2	Crop Disease Intelligence	Disease identification; symptom monitoring; disease severity assessment; disease knowledge
3	Smart Irrigation Intelligence	Soil moisture; rainfall monitoring; crop water requirements; irrigation scheduling
4	Soil and Nutrient Intelligence	Soil properties; nutrient requirements; fertiliser management; soil health monitoring
5	Agricultural Market Intelligence	Price monitoring; supply and demand; market comparison; selling opportunities
6	Dairy Production Intelligence	Milk-yield monitoring; feeding management; animal health; lactation analysis
7	Livestock Health Intelligence	Animal identification; health records; symptom monitoring; veterinary advisory
8	Agricultural Extension Intelligence	Farmer enquiries; agricultural knowledge; expert referral; advisory communication
9	Agricultural Supply Chain Intelligence	Harvest management; grading and sorting; storage; transportation and logistics
10	Smart Greenhouse Intelligence	Environmental monitoring; temperature and humidity; ventilation; automated control

These are proposed course themes, not claims that all ten problems have been validated in Nepal. Students should investigate them during PBL and problem formulation.

Proposed assignment method: each group ranks its top three themes, and the trainers assign them so that every theme is taken by exactly one group [confirm method].

How Each Group Works

Each group works on one shared project, and each of its four members develops one component of it. Group 1 shows the pattern.

Group 1: Crop Pest Intelligence. Main problem: how can farmers improve pest identification, monitoring and crop protection decisions?

Member	Subproblem	What they do	Branch
Member 1	Pest identification	Develops the pest identification subproblem	`member-1-pest-identification`
Member 2	Pest monitoring	Investigates monitoring data and observations	`member-2-pest-monitoring`
Member 3	Crop damage assessment	Analyses the extent and severity of damage	`member-3-damage-assessment`
Member 4	Crop protection knowledge	Studies existing practices and reliable advisory information	`member-4-protection-knowledge`

Integration: all four contributions are combined into one Crop Pest Intelligence project.

The four members are not building four unrelated applications. They are developing four components of one integrated system.

GitHub Architecture

One central repository holds all 10 projects. Each group works on its own branch and maintains its own folder, so the ten systems can eventually be integrated into one project.

Repository

The repository `Agricultural-Agentic-AI` has one branch per group, `group-01` to `group-10`. The `main` branch is the integrated project, where all approved group contributions are merged.

Folder structure

The main branch has this structure:

```
Agricultural-Agentic-AI/
│
├── README.md
├── COURSE.md
│
├── docs/
│   ├── course-architecture.md
│   ├── development-guidelines.md
│   └── integration-guidelines.md
│
├── shared/
```

```
│   ├── common-data/
│   ├── common-utilities/
│   ├── knowledge-resources/
│   └── shared-models/
│
├── groups/
│   ├── group-01-crop-pest/
│   ├── group-02-crop-disease/
│   ├── group-03-irrigation/
│   ├── group-04-soil-nutrient/
│   ├── group-05-market-intelligence/
│   ├── group-06-dairy/
│   ├── group-07-livestock-health/
│   ├── group-08-extension/
│   ├── group-09-supply-chain/
│   └── group-10-smart-greenhouse/
│
├── integration/
│   ├── api/
│   ├── workflows/
│   └── deployment/
│
└── requirements.txt
```

Branch structure

Each group works on its own branch and in its own folder.

Branch	Responsibility	Folder
`main`	Integrated project	All folders
`group-01`	Crop Pest Intelligence	`groups/group-01-crop-pest/`
`group-02`	Crop Disease Intelligence	`groups/group-02-crop-disease/`
`group-03`	Smart Irrigation	`groups/group-03-irrigation/`
`group-04`	Soil and Nutrient	`groups/group-04-soil-nutrient/`
`group-05`	Market Intelligence	`groups/group-05-market-intelligence/`
`group-06`	Dairy Production	`groups/group-06-dairy/`
`group-07`	Livestock Health	`groups/group-07-livestock-health/`
`group-08`	Agricultural Extension	`groups/group-08-extension/`
`group-09`	Supply Chain	`groups/group-09-supply-chain/`
`group-10`	Smart Greenhouse	`groups/group-10-smart-greenhouse/`

Each group organises its own folder into four member contributions:

```
groups/group-01-crop-pest/
│
├── README.md
├── problem-formulation/
├── agricultural-understanding/
├── data/
├── member-01/
├── member-02/
├── member-03/
├── member-04/
├── analytics/
├── rag/
├── agents/
└── integration/
```

How the four members collaborate

The recommended approach is a group branch with member branches. The example below uses Group 1.

```
main
  │
  └── group-01
        │
        ├── member-1-pest-identification
        ├── member-2-pest-monitoring
        ├── member-3-damage-assessment
        └── member-4-protection-knowledge
```

- Group members create their branches from `group-01`.
- Each member works on their assigned subproblem.
- Members submit pull requests into `group-01`.
- The group leader reviews and integrates their work.
- The completed group branch is submitted to `main`.

This creates two levels of integration: individual-to-group and group-to-course.

Development workflow

1. The instructor creates the main repository.
2. Each group creates its group branch from `main`.
3. The four members create individual branches from their group branch.
4. Members develop, commit and submit pull requests to the group branch.
5. The group leader reviews and integrates the four contributions.
6. The group submits a pull request to `main`.
7. The instructor reviews the integrated contribution and merges it.

Important GitHub distinction

A branch does not automatically contain only its own folder. Each branch represents a version of the entire repository. Therefore:

- The group branch contains the whole project at that point, but members should modify only their assigned group folder.
- The main branch receives approved changes from all groups.
- Folder organisation keeps the project structured.
- Branch protection and pull requests control how changes are integrated.

Training Schedule and Learning Progression

Nine stages take the 40 learners from understanding a problem to an evaluated prototype over 60 hours: 40 self-study and 20 workshop.

Stage	Hours (self / workshop)	UDAAN link	Group output
1. Understanding Agriculture/AI	6 / 2	Problem presentation and exploration	`agricultural-understanding/`
2. U0 Problem Formulation	4 / 2	U0: What is the problem?	`problem-formulation/`
3. Infrastructure	2 / 2	Set-up	Working repository and environment
4. U1 Descriptive Analytics	5 / 2	U1: What is happening?	`analytics/`
5. U2 Diagnostic Analytics	5 / 2	U2: Why is it happening?	`analytics/`
6. U3 Predictive Analytics	5 / 2	U3: What might happen next?	`analytics/`
7. RAG, Local LLM and Agent Basics	6 / 2	Build the intelligence	`rag/`, `agents/`
8. Integration, Markdown Workflow, Deployment	5 / 2	U4: What should be done?	`integration/`
9. Capstone and Evaluation	2 / 4	Transform the workflow	Final project and presentation
Total	**40 / 20**		

U3 can be replaced with more U1/U2 and agentic workflow work if suitable historical data is unavailable.

Delivery pattern

All 40 learners do their self-study at the same time, about 5 hours a week over 8 weeks, with light doubt-clearing support. The 20 workshop hours run over 2 weeks at 10 hours a week.

Phase	Duration	Dates
Preparation	[__ weeks]	[start – end]

Phase	Duration	Dates
Self-study	8 weeks, about 5 h/week	[start – end]
Workshops	2 weeks, 10 h/week	[start – end]
Capstone and evaluation	Within the workshop weeks	[date]
Final integration and release	[__ weeks]	[date]

Roles and Governance

The two trainers own the repository and merge to `main`; learners change only their own group's folder.

Role	Who	Responsibility
Trainer (instructor)	2, each supporting 5 groups	Create the repository, protect `main`, review each group's pull request, merge to `main`
Group lead	1 per group, rotating each stage	Reviews and integrates the four member contributions, opens the group pull request
Group member	4 per group	Develops one subproblem on their own branch and submits pull requests to the group branch
Teaching assistant	[if available]	First-pass review of group pull requests

Governance rules:

- Nobody commits directly to `main`; every change arrives through a reviewed pull request.
- Members modify only their assigned group folder.
- Groups record what is reported, what is evidenced, what is assumed and what is unknown. They do not invent figures.
- A learner reviews and validates AI-generated output before it is merged.

Assessment

The course assesses the ability to improve an agricultural workflow, not coding alone, using both the repository record and the capstone.

Component	Level	What is assessed	Weight
Stage deliverables	Group	Quality of each stage's documents and analysis in the group folder	[__%]
Individual contribution	Individual	Commit and pull-request history, reviews given, the member's subproblem	[__%]
Validated analytical report	Group	Tables, charts, findings, limitations, evidence-aware reasoning	[__%]
Capstone prototype	Group	Integrated Agentic AI prototype, testing, safety and usability	[__%]
Capstone evaluation	Group and individual	Evaluation in groups of 3–4 (20–25 minutes each), or two parallel sessions with one trainer each	[__%]

Final Integrated Project

By the end, each group delivers one integrated system, and the trainers merge the ten systems into one course project on `main`.

- Each group combines its four member contributions into one project, for example one Crop Pest Intelligence project for Group 1.
- Each group's folder holds its problem formulation, agricultural understanding document, data, analytics, RAG, agents and integration work.
- Each group presents its project in the capstone evaluation.
- The `integration/` folder holds the shared API, workflows and deployment for the combined project.
- A short summary lists limitations and open questions, using the same evidence rule as the course.

Risks and Open Decisions

The main risk is workshop quality with 40 learners, and six decisions are still open.

Risk	What to do
Hands-on workshops with 40 people are harder than with 20	Add a teaching assistant per cohort, or split the cohort into two groups of 20 for workshops only
Trainers need preparation and grading time beyond contact hours	Plan 3–5 hours a week beyond the 10 contact hours
Self-study is not free	Provide a platform, quizzes, doubt-clearing sessions and progress tracking
Stages 7 and 8 have only 2 workshop hours each for heavy topics	Make the self-study material strong and the workshops hands-on
A venue or platform that cannot hold 40 people	Confirm capacity before day 1
Learners treat the four subproblems as separate applications	Review at each stage that the four contributions fit one system

Open decisions:

- [ ] Can the venue or platform handle 40 people in a workshop?
- [ ] Is a teaching assistant or a third trainer available?
- [ ] Is the 40 self-study / 20 workshop split final?
- [ ] Do workshops run as one block after self-study, or alternate stage by stage?
- [ ] How are groups formed and how are themes assigned (criteria and method)?
- [ ] What are the assessment weights?